

Report of Individual Contributions

Secure Digital Document Vault (SDDV) — UNAM 2026-2

Emilio Sebastián Contreras Colmenero — Equipo 7

Report of Individual Contributions

Autor: Emilio Sebastián Contreras Colmenero Proyecto: Secure Digital Document Vault (SDDV) Materia: Criptografía — Dra. Rocío Aldeco Pérez · UNAM 2026-2 Equipo: Equipo 7 Repositorio: github.com/milliyx/Cryptography Periodo cubierto: febrero 2026 – mayo 2026

1. Mi rol en el equipo

Dentro del Equipo 7 mi rol fue el de implementador del core criptográfico, auditor de seguridad y documentador técnico. En total fui responsable de los tres módulos centrales del rubric (D2, D3, D5), de la auditoría de vulnerabilidades que se hizo al final, y de la mayor parte de los documentos de diseño y de los reportes formales. También terminé asumiendo tareas de mantenimiento del repositorio cuando hubo que refactorizar código común o renombrar entregables para alinear con la numeración del Canvas.

Las áreas técnicas en las que trabajé son básicamente todas las relacionadas con la criptografía simétrica y asimétrica que usa el proyecto: cifrado AEAD con AES-256-GCM y ChaCha20-Poly1305, KDF y gestión de llaves Ed25519, cifrado híbrido KEM+DEM con X25519, y firmas digitales encadenadas sobre contenedores híbridos. Los módulos en los que aporté son:

- `crypto/aead.py` (D2)
 - `crypto/hybrid.py` (D3)
 - `crypto/signatures.py` y `crypto/secure_send.py` (D5)
 - `crypto/keys.py` y `crypto/kdf.py` (apoyé en ambos)
 - `audit_tampering.py`, `audit/vuln1_path_traversal.py`, `audit/vuln2_replay.py` (auditoría)
 - todos los archivos de tests asociados a los módulos anteriores.
-

2. Contribuciones técnicas

2.1 Implementación de los módulos D2, D3 y D5

La primera entrega que hice fue el módulo D2 de cifrado AEAD en `crypto/aead.py`. Aquí implementé el contenedor binario SDDV con su cabecera completa (magic, versión, algoritmo, timestamp, filename) que se usa como AAD del cifrado AES-256-GCM. La decisión de seguridad más importante en esta parte fue que cualquier modificación a los metadatos invalide el tag AEAD, porque así un atacante no puede cambiar el nombre del archivo o el timestamp sin que se note. También implementé el soporte para dos algoritmos (AES-256-GCM y ChaCha20-Poly1305) para que el sistema pudiera elegir el más conveniente según el hardware.

Para el D3 (cifrado híbrido) implementé en `crypto/hybrid.py` el esquema KEM+DEM con X25519, donde cada destinatario tiene su propio par efímero (lo cual da forward secrecy: comprometer una llave de largo plazo no expone los mensajes pasados). El contenedor SDDH lleva todos los `wrapped_key` de cada destinatario como parte del AAD, así que ni la lista de destinatarios se puede manipular sin que se rompa el tag.

Para el D5 implementé `crypto/signatures.py` y `crypto/secure_send.py`, donde la decisión clave fue usar el patrón encrypt-then-sign con verify-first: la firma Ed25519 se hace sobre el contenedor SDDH completo (incluyendo el fingerprint del firmante como binding de identidad), y al recibir se verifica la firma ANTES de descifrar. Esto previene exponer el plaintext si la firma no es válida.

2.2 Auditoría de seguridad y hardening

Esta fue la contribución más diferenciada que aporté. Después de que el cifrado estaba funcionando, en mayo dediqué casi una semana entera a atacar mi propio sistema. Escribí `audit_tampering.py` que es un script que prueba modificaciones sistemáticas al contenedor cifrado y verifica que el sistema las detecte. De ese trabajo salieron 7 vulnerabilidades documentadas (VULN-001 a VULN-007), de las cuales las dos más severas las arreglé yo mismo:

- VULN-001 — Path traversal (CWE-22): el filename del contenedor no estaba siendo validado, así que un atacante podía meter `../../../../etc/passwd` como nombre y al desempaquetar el archivo se escribía fuera del directorio destino. El fix está en el commit `b40d0c0` y se prueba en `audit/vuln1_path_traversal.py`.
- VULN-002 — Replay attack (CWE-294): el timestamp del contenedor no se validaba contra una ventana de freshness, así que un contenedor capturado en el pasado se podía reenviar. El fix está en `6a98cd1` y se prueba en `audit/vuln2_replay.py`.

Las otras cinco vulnerabilidades (VULN-003 a VULN-007) las arreglé en el commit `352919e` como hardening de prioridad alta, y todos los fixes tienen tests de regresión en `tests/test_security_patches.py`. Toda la auditoría está documentada en `docs/vulnerability_report.md` (Final Security Report) que tiene 883 líneas con análisis de cada CVE, evidencia de explotación antes del fix, evidencia de mitigación después, y capturas de terminal.

2.3 Documentación técnica

Escribí o coautoré los documentos de diseño: `docs/D2_Encryption_Design.md` (191 líneas), `docs/D5_Signature_Design.md` (316 líneas), el README principal del proyecto, el guion de presentación, y el Final Security Report. También generé los PDFs `sddv_d4.pdf` y `sddv_d4.pptx` para las defensas.

2.4 Refactor de calidad

El commit `f06bc21` (“promover validators, deduplicar header SDDV/SDDH y reorganizar archivos”) fue un refactor importante donde extraje las validaciones que estaban dispersas en `aead.py` y `hybrid.py` a un módulo público compartido, y deduplico código de serialización de headers que se había copy-pasteado entre los dos contenedores. Después del refactor los 122 tests existentes siguieron pasando, lo cual fue una buena señal de que la cobertura de tests era sólida.

3. Evidencia en GitHub

Mi cuenta de GitHub en este proyecto es `@SEBASTIANCONTRERAS35` (con commits firmados también desde Emilio Sebastian Contreras Colmenero `<emilio3547@outlook.es>` y desde la máquina con alias `BI-CH0TEE`).

Totales en **origin/main**: 17 commits sin merges, +9,470 líneas añadidas y -1,148 quitadas, repartidas en 37 archivos únicos.

3.1 Commits clave

Hash	Fecha	Descripción
2168963	15-feb-2026	Actualizar README con descripción completa del proyecto

Hash	Fecha	Descripción
12b31dd	04-mar-2026	feat(D2): Add AEAD encryption module (crypto/aead.py)
1a39b8a	02-abr-2026	feat: Implementar KDF, gestión de llaves, firmas y cifrado híbrido D3
4ea9182	04-abr-2026	refactor: eliminar Argon2id (no pedido por la profa)
b38cac1	25-abr-2026	feat(D4): firma digital sobre contenedores híbridos (SDDH)
560e93d	25-abr-2026	rename: D4 → D5 para alinear con numeración del Canvas
867516c	05-may-2026	feat: auditoria de seguridad — script de tampering, log y reporte PDF
b40d0c0	05-may-2026	fix(VULN-001): validar filename para prevenir path traversal (CWE-22)
6a98cd1	05-may-2026	fix(VULN-002): validar freshness de timestamp para prevenir replay (CWE-294)
ab0a10c	05-may-2026	docs: agregar Vulnerability Report con estructura formal
f06bc21	07-may-2026	refactor: promover validators, deduplicar header SDDV/SDDH
352919e	08-may-2026	feat(security): hardening de prioridad alta — VULN-003..007

3.2 Pull Requests y ramas

Las ramas principales en las que trabajé y que se mergeraron a main son feature/aead-module (PR #1), feature/d3-hybrid (PRs #2 y #3). En total participé en la mayoría de los PRs del proyecto, ya sea como autor del contenido o como reviewer del trabajo de los compañeros antes del merge.

3.3 Por qué estas contribuciones importan

Cada uno de los módulos que implementé es bloqueante para los demás: sin el D2 no hay D3 (porque el cifrado simétrico de los contenedores SDDH usa el AEAD de D2 por debajo), y sin el D3 no hay D5 (porque la firma se hace sobre el contenedor SDDH completo). La auditoría de seguridad mejoró el sistema en un sentido muy concreto: pasamos de tener vulnerabilidades de path traversal y replay sin detectar, a tener un suite de tests que las detecta y previene en cada CI. Los design docs sirven al equipo para defender el trabajo frente a la profe y para que quien retome el proyecto en el futuro entienda las decisiones de diseño.

4. Colaboración y trabajo en equipo

Mi rol en la colaboración fue una mezcla de implementador independiente, revisor y coordinador del rubric. Por un lado, trabajé bastante solo en mis módulos (D2, D3, D5, audit), pero cada vez que mi trabajo dependía del de alguien más o cuando alguien necesitaba algo de mí, coordinamos activamente.

Donde más colaboré con el resto del equipo fue en:

- Integración con D6 de Evan. Cuando Evan implementó la KeyStore API en crypto/keystore.py, la integré con mi secure_send.py para que las funciones encrypt_and_sign_from_keystore pudieran desbloquear las llaves del keystore sin cachearlas. Antes de eso, mi código recibía objetos de clave directamente, así que tuvimos que acordar el contrato de la API.
- Renombramiento D4 → D5. Cuando la profe pidió alinear la numeración con el Canvas, yo hice el renombre completo (commit 560e93d) actualizando todas las referencias en código, tests, docs y comentarios. Esto evitó que el equipo tuviera mensajes contradictorios entre commits viejos y documentación nueva.

- Generación de entregables. Yo generaba los PDFs y los PPTX que se subían al Classroom (sddv_d4.pdf, sddv_d4.pptx, los reportes de vulnerabilidades, y ahora el D6_Key_Management.pdf). Esta tarea no se ve en commits individuales muy llamativos pero implica revisar que cada entrega cumpla con todos los puntos de la rúbrica.
 - Debugging y review. Revisé varios PRs antes del merge (especialmente cuando tocaban código que yo había escrito, para evitar regresiones) y ayudé a debuggear cuando los tests de algún compañero fallaban por interacciones con mis módulos.
-

5. Reflexión final

¿Cuál fue mi contribución más importante? Sin duda la auditoría de seguridad. Es la contribución que más me costó pero también la que más valor agregó al proyecto, porque transformó el sistema de “funciona en el caso feliz” a “funciona y además resiste ataques conocidos”. Implementar los módulos criptográficos es importante, pero al final esos módulos los podía haber escrito siguiendo la documentación de cryptography; en cambio, encontrar las vulnerabilidades reales en mi propio código requirió cambiar de mentalidad: dejar de pensar como implementador y empezar a pensar como atacante. Los commits de los VULN fixes son los que más orgullo me dan, porque cada uno representa un agujero real que el sistema tenía y que ahora ya no tiene.

¿Cuál fue el reto más difícil? El reto técnico que más me costó fue vincular correctamente los metadatos al tag AEAD como AAD. La idea suena simple: si modificas el filename, el tag debe romperse. Pero la implementación implica que el orden de serialización de los campos sea exactamente el mismo en el cifrado y en el descifrado, byte por byte, porque cualquier diferencia (incluso un endianness mal puesto en el FNAME_LEN) hace que el descifrado siempre falle y no es obvio si el fallo es por manipulación legítima o por un bug. Pasé varias horas debuggeando casos donde el cifrado funcionaba pero el descifrado fallaba porque el AAD que se reconstruía no era idéntico al original. Al final lo resolví con tests muy específicos que comparan el AAD byte por byte.

¿Qué concepto de seguridad entendí mejor? AEAD y por qué importa el AAD. Antes del proyecto pensaba que el cifrado y la autenticación eran cosas separadas (ciframos con AES, después ponemos un MAC). Con AEAD aprendí que es mucho mejor tener un solo mecanismo que cubra los dos, y que el campo de Associated Data es lo que permite proteger metadatos que no son secretos pero sí tienen que ser íntegros. Esta idea se transfiere a muchos otros contextos: el header de un JWT, el frame de TLS, el contenedor de Signal. Una vez que entiendes el patrón AEAD+AAD, lo ves en todas partes.

¿Qué mejoraría a futuro? Tres cosas concretas: (1) automatizar la auditoría de seguridad como parte del CI, en vez de ejecutarla manualmente como hicimos esta vez —que algunos checks corran automáticamente en cada PR; (2) escribir más property-based tests con `hypothesis` para que las manipulaciones aleatorias del contenedor se prueben más allá de los casos que yo pensé; y (3) reducir el tiempo que tarda la integración entre los entregables del equipo, generando documentación de las APIs públicas con antelación para que cada quien sepa exactamente qué firma de función espera el código de los demás.